

Idea Lab: A Self-Service Cross-Disciplinary Team Formation Platform for DBIT

Harsha N¹, Mithun Gowda B², Naren V³, Nevil Anson Dsouza⁴

Department of Computer Science and Engineering, Section B

Don Bosco Institute of Technology, Bangalore, India

¹harsha210108@gmail.com, ²mithungowda.b7411@gmail.com

³narenbhaskar2007@gmail.com, ⁴nevilansonsouza@gmail.com

Abstract—This paper presents Idea Lab, a web-based team formation platform developed for Don Bosco Institute of Technology (DBIT), Bangalore. The platform enables approximately 825 first-year students across 7 engineering branches to self-organize into cross-disciplinary teams of exactly 6 members with real-time constraint validation and administrative oversight. Built using Next.js 16, Firebase Firestore, and a custom OTP-based authentication system, Idea Lab replaces the manual team assignment process with an automated, constraint-enforced, and scalable solution. The system enforces branch diversity constraints, provides real-time notifications, and offers comprehensive admin controls for managing student registrations and team compositions. The platform is deployed at <https://idea-lab-sage.vercel.app/>.

Index Terms—Team Formation, Web Application, Next.js, Firebase, Cross-Disciplinary Teams, Real-Time Validation, OTP Authentication

I. INTRODUCTION

Forming balanced, cross-disciplinary teams from a large student body is a recurring challenge in educational institutions. Manual assignment methods are time-consuming, error-prone, and often fail to account for branch diversity requirements. At Don Bosco Institute of Technology (DBIT), approximately 825 first-year students across 7 engineering branches—CSE, ISE, AIML, ECE, EEE, CV, and ME—need to form teams for interdisciplinary project work in the Idea Lab program.

Idea Lab addresses this challenge by providing a self-service platform where students can register, create teams, invite members, and join existing teams while the system automatically enforces composition constraints in real time.

The key contributions of this work are:

- A real-time team formation system with automated constraint validation
- A custom OTP-based authentication system using the Brevo Email API
- A comprehensive admin dashboard with gate controls and bulk data management
- A production-grade security architecture with JWT sessions, rate limiting, and Firestore security rules

Repository: <https://github.com/harshaxyZ/idea.lab.git>

Live URL: <https://idea-lab-sage.vercel.app/>

II. SYSTEM ARCHITECTURE

A. High-Level Architecture

Idea Lab follows a modern full-stack architecture with server-side rendering and API routes provided by Next.js 16 (App Router), backed by Firebase Firestore as the primary database.

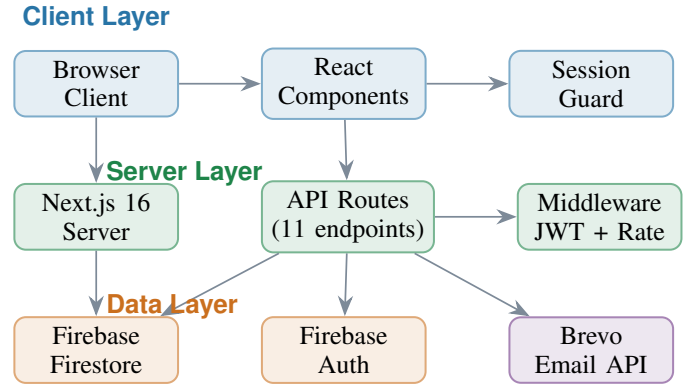


Fig. 1. High-level system architecture of Idea Lab.

B. Firestore Data Model

The system uses 8 Firestore collections as shown in Table I.

TABLE I
FIRESTORE COLLECTIONS AND THEIR PURPOSES

Collection	Purpose
students	CSV-imported master student data
registrations	Active registered students
teams	Team records with member lists
invites	Invite and join request documents
notifications	Real-time user notifications
config	Global configuration flags
otp_codes	Email verification codes
admins	Admin whitelist entries

III. TECHNOLOGY STACK

Table II provides a detailed breakdown.

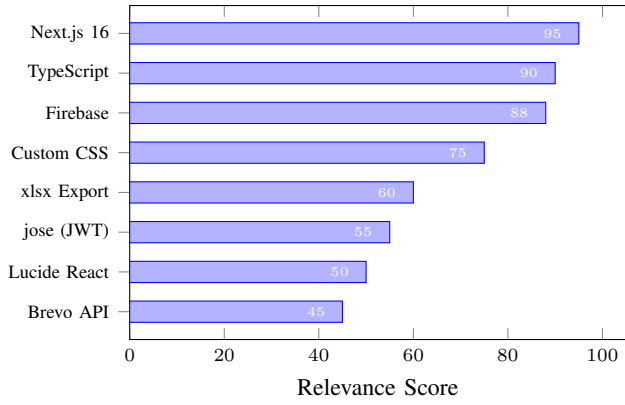


Fig. 2. Technology stack components ranked by relevance.

TABLE II
DETAILED TECHNOLOGY STACK

Layer	Technology
Framework	Next.js 16 (App Router, SSR)
Language	TypeScript 5 (strict mode)
Database	Firebase Firestore (NoSQL)
Auth	Firebase Auth (admin) + Custom OTP
Email	Brevo Email API (multi-key round-robin)
Styling	Custom CSS (Paper & Ink design system)
Session	JWT (HS256, httpOnly cookie)
Export	xlsx library for Excel generation
Icons	Lucide React
Hosting	Vercel (serverless deployment)

IV. KEY FEATURES

A. Student Registration Flow

The registration process follows a three-step flow:

- 1) **USN Entry** — validated against CSV master data via `/api/auth/lookup-usn`
- 2) **OTP Verification** — 6-digit code sent via Brevo Email API with 10-minute expiry, rate-limited to 5 sends per 15 minutes per IP
- 3) **Session Establishment** — JWT cookie + Firebase custom token issued for dual authentication

B. Team Formation with Constraint Enforcement

Teams must satisfy the constraints shown in Table III.

TABLE III
TEAM COMPOSITION CONSTRAINTS

Rule	Value	Enforcement
Team size	Exactly 6	Hard block at 6
Max same branch	4 from one	Blocked at 5th
Min branches	2 unique	Checked at completion
EEE/ECE req.	At least 1	Warning → hard block

C. Admin Dashboard

The admin panel provides six functional tabs: (1) **Dashboard** with real-time statistics, (2) **Students** for CSV upload with batch writes of 450 documents per batch, (3) **Registrations** with searchable, paginated tables supporting CSV/XLS export, (4) **Teams** for filtered team views, (5) **Admins** for whitelist management, and (6) **Settings** for gate controls and database reset.

V. CODEBASE ANALYSIS

A. Project Statistics

The project comprises **58 TypeScript/TSX files** totaling **10,937 lines of code**.

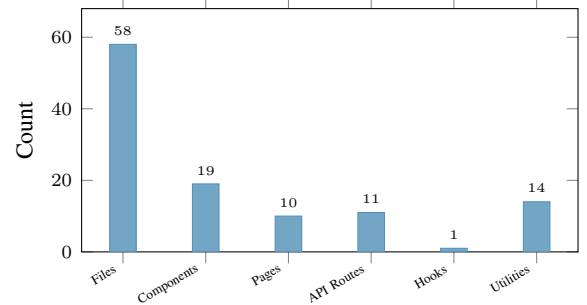


Fig. 3. Codebase composition by category.

B. Source File Distribution

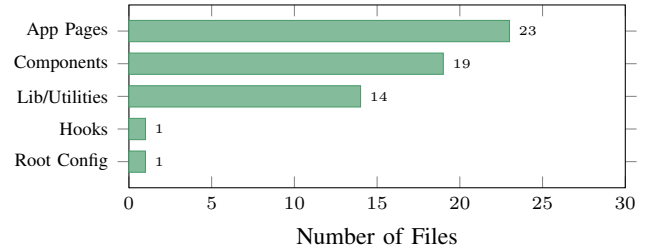


Fig. 4. Source file distribution by directory.

C. File Type Breakdown

The codebase contains **31 TSX files** (React components and pages) and **29 TS files** (logic, utilities, API routes, and configuration).

VI. DEVELOPMENT TIMELINE

The project was developed over a 5-day sprint from March 11–15, 2026, producing **62 commits**.

A. Contribution Distribution

VII. SECURITY ARCHITECTURE

A. Authentication Layers

The system implements a multi-layered authentication approach as illustrated in Fig. 8.

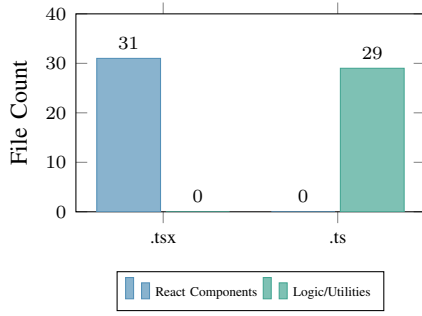


Fig. 5. TypeScript file type distribution.

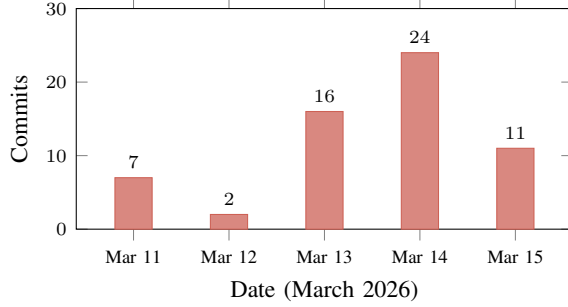


Fig. 6. Commit activity during the development sprint.

B. Rate Limiting

C. Security Headers

The application enforces security headers via Next.js middleware: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Strict-Transport-Security (HSTS), Content-Security-Policy (CSP), and Permissions-Policy (camera, microphone, and geolocation disabled).

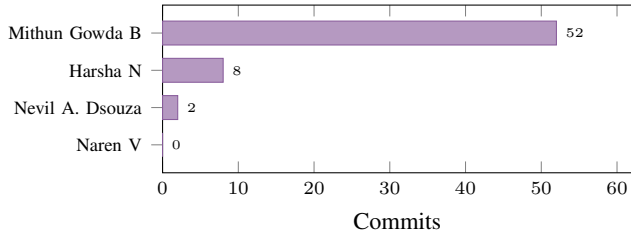


Fig. 7. Team contribution distribution by commits.

TABLE IV
API RATE LIMITING CONFIGURATION

Endpoint	Limit	Window
POST send-otp	5 req/IP	15 min
POST verify-otp	10 att/IP	15 min
Per-OTP attempts	5 attempts	Until expiry
Per-email cooldown	1 OTP	60 sec

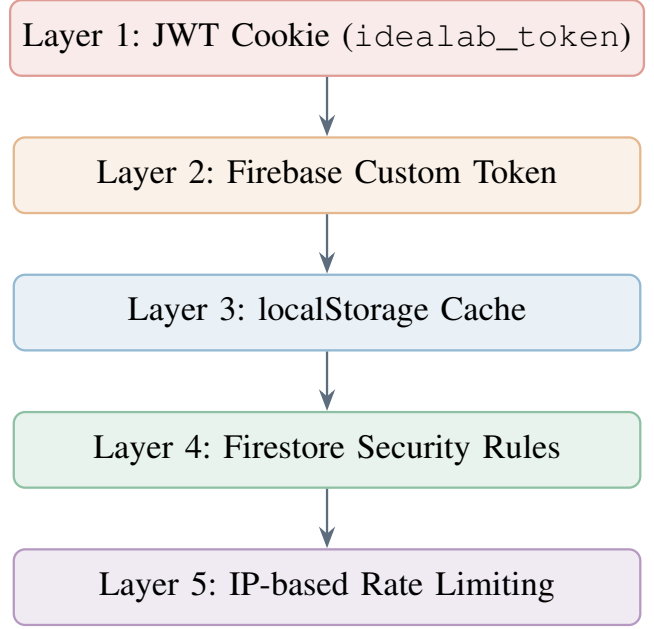


Fig. 8. Multi-layered security architecture.

VIII. USER FLOW ANALYSIS

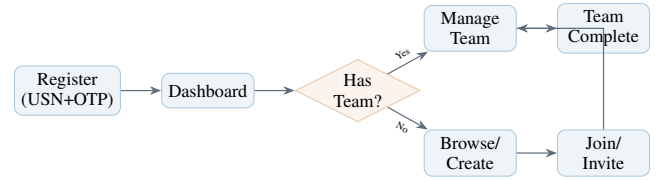


Fig. 9. Student user flow through the platform.

IX. TEAM DETAILS

TABLE V
PROJECT TEAM MEMBERS

#	Name	USN	Email
1	Harsha N	1DB25CS065	harsha210108
2	Mithun Gowda B	1DB25CS109	mithungowda.b7411
3	Naren V	1DB25CS113	narenbaskar2007
4	Nevil Anson Dsouza	1DB25CS116	nevilansondsouza

All emails use the @gmail.com domain.

All team members are first-year B.E. students in the **Department of Computer Science and Engineering (CSE), Section B** at **Don Bosco Institute of Technology (DBIT)**, Bangalore, affiliated to Visvesvaraya Technological University (VTU).

A. Individual Contributions

- **Mithun Gowda B** — Lead developer. Implemented the core architecture, authentication system (JWT + OTP), team constraint engine, admin dashboard, API routes,

TABLE VI
TEAM MEMBER CONTACT DETAILS

Name	Phone	Branch	Sec.
Harsha N	7899214458	CSE	B
Mithun Gowda B	9844556496	CSE	B
Naren V	7204375655	CSE	B
Nevil Anson Dsouza	9980445460	CSE	B

middleware, security layers, and deployment. Authored 52 of 62 commits (84%).

- **Harsha N** — Contributed to the Brevo OTP authentication integration, invite system enhancements, and pull request workflows. Authored 8 commits (13%).
- **Nevil Anson Dsouza** — Contributed to admin panel functionality and server-side API route migrations. Authored 2 commits (3%).
- **Naren V** — Participated in code review, testing, and quality assurance throughout the development cycle.

X. DEPLOYMENT & INFRASTRUCTURE

TABLE VII
DEPLOYMENT CONFIGURATION

Component	Details
Hosting	Vercel (serverless)
Database	Firebase Firestore (Google Cloud)
Email	Brevo API (300 emails/day free tier)
Domain	idea-lab-sage.vercel.app
Repository	github.com/harshaxyZ/idea.lab
Build	Next.js 16
CI/CD	Vercel Git Integration (auto-deploy)

XI. DESIGN SYSTEM

The platform uses a custom design system called “Paper & Ink” with a minimal, print-inspired aesthetic built on four core colors:



Fig. 10. Paper & Ink design system color palette.

XII. CHALLENGES & SOLUTIONS

XIII. FUTURE WORK

- Integration with university ERP systems for automatic student data synchronization
- Real-time collaborative team workspace with shared documents
- AI-powered team recommendation engine based on student skills and interests

TABLE VIII
KEY CHALLENGES AND SOLUTIONS

Challenge	Solution
Email rate limits (300/day)	Multi-key round-robin rotation
Race conditions in team joins	Firestore transactions + server-side validation
SSR hydration mismatches	SessionGuard with client-only rendering
Cross-origin API abuse	Origin-checking middleware
Large CSV uploads	Batch writes (450 docs/batch)

- Mobile application for improved accessibility
- Analytics dashboard with team performance metrics

XIV. CONCLUSION

Idea Lab demonstrates that a self-service, constraint-enforced team formation platform can effectively replace manual processes at scale. The system handles 825+ students across 7 branches with real-time validation, ensuring every team meets diversity requirements. The 5-day development sprint produced a production-grade application with 10,937 lines of TypeScript, 19 reusable components, 11 API routes, and comprehensive security layers. The platform is deployed at <https://idea-lab-sage.vercel.app/>, with source code at <https://github.com/harshaxyZ/idea.lab.git>.

REFERENCES

- [1] Vercel, “Next.js Documentation,” 2026. [Online]. Available: <https://nextjs.org/docs>
- [2] Google, “Firebase Documentation,” 2026. [Online]. Available: <https://firebase.google.com/docs>
- [3] Brevo, “Email API Documentation,” 2026. [Online]. Available: <https://developers.brevo.com>
- [4] IETF, “RFC 7519 — JSON Web Token (JWT),” May 2015.
- [5] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [6] Google, “Cloud Firestore Security Rules,” 2026. [Online]. Available: <https://firebase.google.com/docs/firestore/security/get-started>